

Documentation : Installer et configurer un serveur SSH (OpenSSH)



Sommaire :

- 1 - Qu'est ce que SSH et OpenSSH
- 2 - Installation du service OpenSSH
- 3 – Configuration simple du service SSH
- 4 - Lancer le service au démarrage
 - 4-1 Contrôle de l'écoute/Listen réseau (netstat/ss)
 - 4-2 Contrôle des logs (journalctl ...)
- 5 - Configuration avancée du service SSH
 - 5-1 Désactiver les connexions SSH en root
 - 5-2 Changer le port de SSH
 - 5-3 Autoriser/Interdire des utilisateurs
 - 5-4 Autoriser/Interdire des groupes

5-5 Afficher une bannière

5-6 Directive Match

5-7 Restriction des droits des utilisateurs (su/sudo)

6 - Tester la connexion ssh :

6-1 En local

6-2 A distance

6-3 Contrôle des journaux de bord (log)

7 – Modification des clés de cryptage

7-1 Générer la nouvelle paire de clés

7-2 Configurer le service SSH

7-3 Aperçu du cryptage via une capture réseau (wireshark)

8 - Infos pratiques côté Client

8-1 ~/.ssh/config : Configuration simple

8-2 ~/.ssh/config : Configuration avancée

1 – Introduction

La sécurisation des communications à distance est un enjeu majeur dans l'administration des systèmes informatiques. SSH (*Secure Shell*) est un protocole essentiel qui permet d'établir une connexion sécurisée entre un client et un serveur, garantissant ainsi la confidentialité et l'intégrité des données échangées.

Dans cet article, nous allons voir comment installer et configurer un serveur SSH en utilisant **OpenSSH**, l'une des implémentations les plus populaires du protocole. Nous aborderons les étapes clés de l'installation, la configuration du service, ainsi que les bonnes pratiques de sécurisation pour éviter les accès non autorisés. Que vous soyez administrateur système ou simple utilisateur souhaitant gérer un serveur à distance, cette guide vous aidera à mettre en place un accès SSH fiable et sécurisé.

Qu'est-ce que SSH ?

SSH (*Secure Shell*) est un protocole réseau permettant d'établir une connexion sécurisée entre un client et un serveur. Il est principalement utilisé pour l'administration à distance des systèmes, l'exécution de commandes et le transfert sécurisé de fichiers. Contrairement aux protocoles non sécurisés comme FTP, SSH chiffre les communications, empêchant ainsi toute interception ou modification des données par des attaquants.

Qu'est-ce qu'OpenSSH ?

OpenSSH (*Open Secure Shell*) est une suite d'outils open-source qui implémente le protocole SSH. OpenSSH est l'une des solutions SSH les plus utilisées sous Linux. Il offre plusieurs fonctionnalités, notamment :

- **ssh** : client SSH pour se connecter à un serveur distant

- **sshd** : serveur SSH qui écoute les connexions entrantes
- **scp** : outil pour le transfert sécurisé de fichiers
- **sftp** : protocole sécurisé pour le transfert de fichiers interactif
- **ssh-keygen** : générateur de clés SSH pour l'authentification par clé publique

2 - Installation du serveur OpenSSH

Pour installer un serveur ssh sur une machine Linux (Debian), il faut installer le paquet openssh-server.

```
apt install openssh-server
```

Résultat :

```
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
...
Installation terminée.
```

Le fichier de configuration principal d'OpenSSH est :

- `/etc/ssh/sshd_config` : Ce fichier contient tous les paramètres du serveur SSH.

Après chaque modification de ce fichier, il est nécessaire de redémarrer le service SSH pour appliquer les changements :

```
sudo systemctl restart ssh
```

4 - Lancer le service au démarrage

Cela permet de garantir un accès à distance permanent au serveur, facilitant l'administration et la gestion en cas de redémarrage. Cela permet aussi d'assurer la continuité des services nécessitant SSH.

```
systemctl enable --now ssh.service
```

Puis vérifié l'installation

Après l'installation, vérifions que le service SSH fonctionne :

```
systemctl status ssh
```

Si le serveur est actif, vous verrez une sortie similaire à :

Résultat :

- ssh.service - OpenBSD Secure Shell server

Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)

Active: active (running) since ...

Les fichiers de configuration principaux sont :

- /etc/ssh/sshd_config : Configuration du serveur SSH
- ~/.ssh/ : Dossier contenant les clés et configurations spécifiques à l'utilisateur

Un redémarrage du service est nécessaire après modification du fichier :

```
sudo systemctl restart ssh
```

4-1 Contrôle de l'écoute/Listen réseau (netstat/ss)

Contrôle de l'écoute réseau (netstat/ss)

```
netstat -tlnp | grep ssh
```

Résultat :

```
tcp    0    0.0.0.0:22    0.0.0.0:*    LISTEN    1234/sshd
```

Voici une commande essentielle pour diagnostiquer et sécuriser un serveur SSH en vérifiant son état et son port d'écoute. Elle permet de vérifier si le service SSH est en écoute sur un port et sur quelle interface il fonctionne. Elle est très utile pour le diagnostic réseau et la sécurité du serveur.

```
ss -tlnp | grep ssh
```

Résultat :

```
LISTEN 0      128    0.0.0.0:22    0.0.0.0:*    users:(("sshd",pid=1234,fd=3))
```

4-2 Contrôle des logs (journalctl ...)

Pour consulter les logs du service SSH et détecter d'éventuelles erreurs :

```
journal -u ssh.service --since "1 hour ago"
```

Résultat :

```
Mar 26 14:00:00 serveur systemd[1]: Started OpenSSH server daemon.  
Mar 26 14:05:10 serveur sshd[2345]: Accepted password for user from 192.168.1.10 port 54321  
ssh2
```

```
Include /etc/ssh/sshd_config.d/*.conf
```

```
PermitRootLogin no  
PermitRootLogin prohibit-password  
Port 2022  
ListenAddress 0.0.0.0  
ListenAddress ::
```

Puis redémarrer SSH :

```
sudo systemctl restart ssh
```

5-2 Changer le port de SSH

Changer le port SSH par défaut (22) permet de réduire les tentatives d'attaques automatisées et les scans de ports malveillants, car de nombreux bots ciblent en priorité ce port bien connu.

Modifier /etc/ssh/sshd_config :

```
include /etc/ssh/sshd_config.d/*.conf
```

```
PermitRootLogin yes  
PermitRootLogin prohibit-password  
Port 2022  
ListenAddress 0.0.0.0  
ListenAddress ::
```

Appliquer la modification :

```
sudo systemctl restart ssh
```

Vérifier :

```
ss -tlnp | grep ssh
```

5-3 Autoriser/Interdire des utilisateurs

Cela permet de **restreindre l'accès SSH** à une liste spécifique d'utilisateurs. Seulement les utilisateurs **user1**, **user2** pourront se connecter via SSH, tandis que tous les autres comptes comme le compte user3 seront bloqués, améliorant ainsi la sécurité du serveur.

Dans `/etc/ssh/sshd_config`, ajouter :

```
AllowUsers user1 user2  
DenyUsers user3
```

Redémarrer SSH :

```
sudo systemctl restart ssh
```

5-4 Autoriser/Interdire des groupes

Dans un environnement multi-utilisateurs, il est important de contrôler qui peut accéder au serveur via SSH. Plutôt que de gérer les autorisations utilisateur par utilisateur, il est plus efficace d'utiliser des groupes.

Si vous voulez restreindre l'accès SSH à un ensemble d'utilisateurs bien défini (par exemple, les administrateurs ou les développeurs), vous pouvez spécifier un groupe dans `/etc/ssh/sshd_config` :

```
AllowGroups sshusers  
DenyGroups nogroup
```

Redémarrer SSH :

```
sudo systemctl restart ssh
```

5-5 Afficher une bannière

Une bannière SSH peut servir d'avertissement légal pour informer les utilisateurs qu'ils accèdent à un système sécurisé et que toute tentative d'accès non autorisé est interdite, afficher une bannière avant l'accès SSH peut-être obligatoire pour respecter certaines normes de cybersécurité ou encore afficher des instructions spécifiques.

Créer une bannière dans `/etc/issue.net` :

Bienvenue sur ce serveur sécurisé.

Accès interdit aux utilisateurs non autorisés !

Toute activité est surveillée et enregistrée.

Puis modifier `/etc/ssh/sshd_config` :

```
Banner /etc/issue.net
```

Redémarrer SSH :

```
sudo systemctl restart ssh
```

5-6 Directive Match

La directive Match permet d'appliquer des règles spécifiques en fonction de l'utilisateur, du groupe, de l'adresse IP ou même du type de connexion. Cela permet de personnaliser la configuration SSH de manière flexible et granulaire.

```
Match User admin
```

```
    PasswordAuthentication no
```

Ici, permet de personnaliser des règles pour certains utilisateurs :

```
Match User user1
    X11Forwarding no
    AllowTcpForwarding no
```

Redémarrer SSH :

```
sudo systemctl restart ssh
```

5-7 Restriction des droits des utilisateurs (su/sudo)

Pour interdire un utilisateur d'utiliser su :

```
sudo usermod -aG nosu user1
```

Puis ajouter dans /etc/pam.d/su :

```
auth required pam_wheel.so group=nosu
```

6-1 En local

Pour tester SSH sur la même machine :

```
ssh localhost
```

6-2 À distance

Pour se connecter à un serveur distant :

```
ssh user@ip_du_serveur -p 2222
```

Remplacez ip_du_serveur par l'adresse IP ou le nom de domaine du serveur.

7-1 Générer la nouvelle paire de clés

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa
```

7-2 Configurer le service SSH

Copier la clé publique vers le serveur pour éviter de saisir le mot de passe à chaque connexion :

```
ssh-copy-id user@serveur
```

Lancer Wireshark et filtrer :

```
tcp.port == 22
```

Observer les échanges chiffrés entre le client et le serveur.

Host serveur

HostName 192.168.1.10

User user1

Port 2222

Host serveur

HostName 192.168.1.10

User user1

Port 2222

IdentityFile ~/.ssh/id_rsa

Compression yes

ForwardAgent yes

L'installation et la configuration d'OpenSSH sur Debian permettent d'assurer un accès sécurisé à un serveur distant. Nous avons vu comment installer OpenSSH, le configurer pour renforcer la sécurité et tester les connexions locales et distantes. Les options avancées, telles que la restriction des utilisateurs, le changement de port et l'ajout de bannières, contribuent à limiter les risques d'intrusion. L'analyse des journaux et la gestion des clés de chiffrement renforcent encore la protection du serveur. En maîtrisant ces aspects, même un débutant peut sécuriser efficacement ses accès SSH et administrer son serveur en toute sérénité.

Problème : Le client affiche "Connection refused". **Solution :**

Vérifier si le service SSH est bien actif avec :

```
sudo systemctl status ssh
```

Redémarrer le service si nécessaire :

```
sudo systemctl restart ssh
```

Vérifier si le pare-feu bloque la connexion :

```
sudo ufw allow ssh  
sudo ufw reload
```

Problème : La connexion met du temps à s'établir. **Solution :**

Modifier le fichier `/etc/ssh/sshd_config` et ajouter :

```
UseDNS no
```

Redémarrer le service :

```
sudo systemctl restart ssh
```

Problème : Impossible de se connecter avec l'utilisateur root. **Solution :**

Modifier le fichier `/etc/ssh/sshd_config` et s'assurer que la ligne suivante est bien présente :

```
PermitRootLogin yes
```

Appliquer les changements :

```
sudo systemctl restart ssh
```

Problème : Erreur "Permission denied (publickey)". **Solution :**

Vérifier que la clé publique est bien ajoutée au fichier `~/.ssh/authorized_keys` sur le serveur.

Vérifier les permissions :

```
chmod 700 ~/.ssh
```

```
chmod 600 ~/.ssh/authorized_keys
```